

```

/*
 * Micromouse Maze Solver – mbed firmware
 * -----
 * Runs a classic three-phase micromouse strategy on an 8x8 maze:
 *
 * 1. EXPLORE   - feel your way to the centre, learning the wall layout
 *                as you go (flood-fill re-run after every new wall).
 * 2. RETURN    - head back to the start using what you've learned.
 * 3. SPEEDRUN  - now that the map is known, plan the shortest path
 *                once and drive it without stopping to think.
 *
 * Hardware: 7 IR wall sensors on a BusIn, two stepper motors driven
 * through L297 controllers on a BusOut, and 4 onboard LEDs used as a
 * cheap "debugger" since there's no serial console on the track.
 */

#include "mbed.h"

// -----
// Maze geometry
// -----
#define MAZE_W 8
#define MAZE_H 8
#define MAX_PATH (MAZE_W * MAZE_H)    // A path can never be longer than every cell in the maze
#define DIST_UNREACHABLE 255          // Sentinel: "flood fill hasn't reached this cell yet"

#define START_X 0
#define START_Y 0

// The centre of an 8x8 maze is a 2x2 block of cells, not a single cell,
// so floodFillGoal() seeds all four. GOAL_X/GOAL_Y just anchor the
// "did we arrive" check to one of them.
#define GOAL_X 3
#define GOAL_Y 3

#define WALL_NORTH (1 << 0)
#define WALL_EAST  (1 << 1)
#define WALL_SOUTH (1 << 2)
#define WALL_WEST  (1 << 3)

// -----
// Motor timing
// -----
#define STEP_DELAY_US    2000    // Time between clock pulses to the L297s
#define STEPS_PER_CELL   512     // Steps to advance exactly one maze cell
#define STEPS_PER_90     256     // Steps for a ~90 degree turn on the spot
#define STEPS_PER_180    512     // Steps for a ~180 degree turn on the spot

#define DIR_LEFT_FWD  1
#define DIR_LEFT_BWD  0
#define DIR_RIGHT_FWD 0
#define DIR_RIGHT_BWD 1

// -----
// Sensor bit layout (BusIn is active-low: 0 = wall seen)
// -----
#define SENSOR_RR_BIT 0    // right-rear
#define SENSOR_RC_BIT 1    // right-centre
#define SENSOR_RL_BIT 2    // right-inner
#define SENSOR_F_BIT  3    // front
#define SENSOR_LR_BIT 4    // left-inner
#define SENSOR_LC_BIT 5    // left-centre
#define SENSOR_LL_BIT 6    // left-rear

#define SENSOR_RIGHT_GROUP_MASK 0x07    // bits 0-2: right-side sensors

```

```

#define SENSOR_FRONT_GROUP_MASK 0x08    // bit 3: front sensor
#define SENSOR_LEFT_GROUP_MASK  0x70    // bits 4-6: left-side sensors

// -----
// LED bus order (used purely for on-track debugging)
// -----
#define LED_MOVE   0x01    // lit while driving or turning
#define LED_WALL   0x02    // pulses when a wall/correction is detected
#define LED_GOAL   0x04    // lit once the goal or run is reached
#define LED_ERROR  0x08    // lit if the robot gets stuck or finds no path

// -----
// Motor bus order
// -----
#define MOTOR_LEFT_DIR_MASK  0x01
#define MOTOR_LEFT_CLK_MASK  0x02
#define MOTOR_RIGHT_DIR_MASK 0x04
#define MOTOR_RIGHT_CLK_MASK 0x08
#define MOTOR_CLK_MASK (MOTOR_LEFT_CLK_MASK | MOTOR_RIGHT_CLK_MASK)
#define MOTOR_DIR_MASK (MOTOR_LEFT_DIR_MASK | MOTOR_RIGHT_DIR_MASK)

// -----
// Types
// -----

typedef enum {
    NORTH = 0,
    EAST  = 1,
    SOUTH = 2,
    WEST  = 3
} Direction;

typedef enum {
    PHASE_EXPLORE = 0,    // find a way to the centre
    PHASE_RETURN,         // head back to the start
    PHASE_SPEEDRUN,       // drive the best known path
    PHASE_COMPLETE        // done – sit still
} Phase;

// What the current sensor reading suggests we should do next.
typedef enum {
    SENSOR_ACTION_FORWARD = 0,
    SENSOR_ACTION_TURN_RIGHT,
    SENSOR_ACTION_TURN_LEFT,
    SENSOR_ACTION_CORRECT_RIGHT,    // drifting left, nudge right
    SENSOR_ACTION_CORRECT_LEFT,     // drifting right, nudge left
    SENSOR_ACTION_CROSS_JUNCTION,
    SENSOR_ACTION_U_TURN,
    SENSOR_ACTION_UNKNOWN
} SensorAction;

typedef struct {
    int x;
    int y;
} Cell;

// -----
// Hardware
// -----
BusIn  sensors(p5, p6, p7, p8, p15, p16, p17);    // 7 IR wall sensors
BusOut motorBus(p21, p22, p23, p24);             // L297 direction + clock lines
BusOut statusLEDs(LED1, LED2, LED3, LED4);       // onboard debug LEDs

// -----
// Global state

```

```

// -----
unsigned char walls[MAZE_H][MAZE_W]; // known walls per cell (bitmask)
unsigned char dist[MAZE_H][MAZE_W]; // flood-fill distance per cell

Direction heading = NORTH; // which way we currently think we're facing
Phase phase = PHASE_EXPLORE;

int currentX = START_X;
int currentY = START_Y;

unsigned char motorState = 0;
unsigned char ledState = 0;

const int DX[4] = { 0, 1, 0, -1 }; // x delta for N, E, S, W
const int DY[4] = { 1, 0, -1, 0 }; // y delta for N, E, S, W
const unsigned char WALL_MASK[4] = { WALL_NORTH, WALL_EAST, WALL_SOUTH, WALL_WEST };

void stepBoth(int leftDirection, int rightDirection, int steps);

// -----
// LEDs
// -----

void applyLEDState(void)
{
    statusLEDs = ledState;
}

void ledsOff(void)
{
    ledState = 0;
    applyLEDState();
}

void ledOn(unsigned char mask)
{
    ledState |= mask;
    applyLEDState();
}

void ledOff(unsigned char mask)
{
    ledState &= (unsigned char)(~mask);
    applyLEDState();
}

void ledPulse(unsigned char mask, int microseconds)
{
    ledOn(mask);
    wait_us(microseconds);
    ledOff(mask);
}

// -----
// Sensors
// -----

// Sensors are active-low, so a 0 bit means "wall present".
int sensorWall(unsigned int bitNumber)
{
    int value = sensors.read();
    return ((value & (1 << bitNumber)) == 0) ? 1 : 0;
}

int frontWallSensor(void) { return sensorWall(SENSOR_F_BIT); }

```

```

int leftWallSensor(void) { return sensorWall(SENSOR_LC_BIT); }
int rightWallSensor(void) { return sensorWall(SENSOR_RC_BIT); }

// Reads all 7 sensors and flips them so 1 = wall, which is much easier
// to reason about than the raw active-low bus value.
unsigned char readWallPattern(void)
{
    int raw = sensors.read();
    return (unsigned char)((~raw) & 0x7F);
}

int leftGroupDetected(unsigned char pattern) { return (pattern & SENSOR_LEFT_GROUP_MASK) != 0; }
int frontGroupDetected(unsigned char pattern) { return (pattern & SENSOR_FRONT_GROUP_MASK) != 0; }
int rightGroupDetected(unsigned char pattern) { return (pattern & SENSOR_RIGHT_GROUP_MASK) != 0; }

// Maps a raw 7-sensor reading to a high-level action. The magic
// hex values come from bench-testing the sensor array against known
// wall configurations – see the sensor calibration notes in the repo.
SensorAction classifySensorPattern(unsigned char pattern)
{
    if (pattern == 0x22) {
        return SENSOR_ACTION_FORWARD;
    }

    if (pattern == 0x60 || pattern == 0x40 || pattern == 0x20 ||
        pattern == 0x68 || pattern == 0x48 || pattern == 0x28) {
        return SENSOR_ACTION_TURN_RIGHT;
    }

    if (pattern == 0x03 || pattern == 0x01 || pattern == 0x02) {
        return SENSOR_ACTION_TURN_LEFT;
    }

    if (pattern == 0x33 || pattern == 0x11) {
        return SENSOR_ACTION_CORRECT_RIGHT;
    }

    if (pattern == 0x66 || pattern == 0x44) {
        return SENSOR_ACTION_CORRECT_LEFT;
    }

    if (pattern == 0x00) {
        return SENSOR_ACTION_CROSS_JUNCTION;
    }

    if (pattern == 0x2A || pattern == 0x6E || pattern == 0x4C ||
        pattern == 0x3B || pattern == 0x18) {
        return SENSOR_ACTION_U_TURN;
    }

    return SENSOR_ACTION_UNKNOWN;
}

void showSensorActionOnLEDs(SensorAction action)
{
    if (action == SENSOR_ACTION_TURN_RIGHT || action == SENSOR_ACTION_TURN_LEFT ||
        action == SENSOR_ACTION_U_TURN) {
        ledPulse(LED_WALL, 30000);
    } else if (action == SENSOR_ACTION_UNKNOWN) {
        ledPulse(LED_ERROR, 30000);
    }
}

// Small in-cell steering nudge to keep the robot centred between walls.
// NOTE: the original version pulsed both motors identically for either

```

```

// correction, which meant it never actually steered – just added a
// short forward creep. Fixed here so each case nudges the opposite way.
void applySensorCorrection(SensorAction action)
{
    const int NUDGE_STEPS = 20;

    if (action == SENSOR_ACTION_CORRECT_RIGHT) {
        // Drifting left – ease right by slowing the right wheel briefly.
        stepBoth(DIR_LEFT_FWD, DIR_RIGHT_BWD, NUDGE_STEPS);
    } else if (action == SENSOR_ACTION_CORRECT_LEFT) {
        // Drifting right – ease left by slowing the left wheel briefly.
        stepBoth(DIR_LEFT_BWD, DIR_RIGHT_FWD, NUDGE_STEPS);
    }
}

// -----
// Direction helpers
// -----

Direction turnLeft(Direction d)    { return (Direction)((((int)d + 3) % 4); }
Direction turnRight(Direction d)   { return (Direction)((((int)d + 1) % 4); }
Direction turnAround(Direction d)  { return (Direction)((((int)d + 2) % 4); }

// Works out which compass direction you'd face to move from one cell
// to an adjacent one. Assumes the two cells really are neighbours.
Direction cellToDirection(Cell from, Cell to)
{
    int dx = to.x - from.x;
    int dy = to.y - from.y;

    if (dy == 1) return NORTH;
    if (dx == 1) return EAST;
    if (dy == -1) return SOUTH;
    if (dx == -1) return WEST;

    return NORTH; // cells weren't adjacent – shouldn't happen, but fail safe
}

// -----
// Motors
// -----

void applyMotorState(void)
{
    motorBus = motorState;
}

void motorStop(void)
{
    motorState = 0;
    applyMotorState();
}

void setMotorDirections(int leftDirection, int rightDirection)
{
    motorState &= (unsigned char)(~MOTOR_DIR_MASK);

    if (leftDirection) motorState |= MOTOR_LEFT_DIR_MASK;
    if (rightDirection) motorState |= MOTOR_RIGHT_DIR_MASK;

    applyMotorState();
}

void pulseMotorClocks(void)
{

```

```

    motorState |= MOTOR_CLK_MASK;
    applyMotorState();
    wait_us(STEP_DELAY_US);

    motorState &= (unsigned char)(~MOTOR_CLK_MASK);
    applyMotorState();
    wait_us(STEP_DELAY_US);
}

void stepBoth(int leftDirection, int rightDirection, int steps)
{
    setMotorDirections(leftDirection, rightDirection);

    for (int i = 0; i < steps; i++) {
        pulseMotorClocks();
    }
}

void driveForward(void)
{
    ledOn(LED_MOVE);
    stepBoth(DIR_LEFT_FWD, DIR_RIGHT_FWD, STEPS_PER_CELL);
    ledOff(LED_MOVE);
}

void rotateLeft90(void)
{
    ledOn(LED_MOVE);
    stepBoth(DIR_LEFT_BWD, DIR_RIGHT_FWD, STEPS_PER_90);
    ledOff(LED_MOVE);
    heading = turnLeft(heading);
}

void rotateRight90(void)
{
    ledOn(LED_MOVE);
    stepBoth(DIR_LEFT_FWD, DIR_RIGHT_BWD, STEPS_PER_90);
    ledOff(LED_MOVE);
    heading = turnRight(heading);
}

void rotate180(void)
{
    ledOn(LED_MOVE);
    stepBoth(DIR_LEFT_FWD, DIR_RIGHT_BWD, STEPS_PER_180);
    ledOff(LED_MOVE);
    heading = turnAround(heading);
}

// Turns however far is needed (0, 90 left, 90 right, or 180) to face
// the requested compass direction.
void rotateToDirection(Direction target)
{
    int diff = ((int)target - (int)heading + 4) % 4;

    if (diff == 1)    rotateRight90();
    else if (diff == 3) rotateLeft90();
    else if (diff == 2) rotate180();
    // diff == 0 means we're already facing the right way – nothing to do.
}

// -----
// Maze map
// -----

```

```

void initialiseMazeArrays(void)
{
    for (int y = 0; y < MAZE_H; y++) {
        for (int x = 0; x < MAZE_W; x++) {
            walls[y][x] = 0;
            dist[y][x] = DIST_UNREACHABLE;
        }
    }
}

// Records a wall on both sides at once: the cell we're in, and the
// neighbour across that wall (if there is one). Keeps the map consistent
// so flood fill never has to guess.
void setWallBidirectional(int x, int y, Direction dir)
{
    if (x < 0 || x >= MAZE_W || y < 0 || y >= MAZE_H) {
        return;
    }

    int d = (int)dir;
    walls[y][x] |= WALL_MASK[d];

    int nx = x + DX[d];
    int ny = y + DY[d];

    if (nx < 0 || nx >= MAZE_W || ny < 0 || ny >= MAZE_H) {
        return; // edge of the maze - no neighbour to update
    }

    Direction opposite = turnAround(dir);
    walls[ny][nx] |= WALL_MASK[(int)opposite];
}

int canMove(int x, int y, Direction dir)
{
    if (x < 0 || x >= MAZE_W || y < 0 || y >= MAZE_H) {
        return 0;
    }

    int d = (int)dir;

    if (walls[y][x] & WALL_MASK[d]) {
        return 0;
    }

    int nx = x + DX[d];
    int ny = y + DY[d];

    if (nx < 0 || nx >= MAZE_W || ny < 0 || ny >= MAZE_H) {
        return 0;
    }

    return 1;
}

// -----
// Flood fill (Lee's algorithm)
// -----

void resetDistances(void)
{
    for (int y = 0; y < MAZE_H; y++) {
        for (int x = 0; x < MAZE_W; x++) {
            dist[y][x] = DIST_UNREACHABLE;
        }
    }
}

```

```

    }
}

// Breadth-first expansion from whatever cells are already queued
// (and already have dist == 0). Shared by both the "flood to goal"
// and "flood to start" variants below.
void runFloodFromQueue(Cell queue[], int back)
{
    int front = 0;

    while (front < back) {
        Cell c = queue[front++];

        for (int d = 0; d < 4; d++) {
            if (!canMove(c.x, c.y, (Direction)d)) {
                continue;
            }

            int nx = c.x + DX[d];
            int ny = c.y + DY[d];

            if (dist[ny][nx] == DIST_UNREACHABLE) {
                dist[ny][nx] = dist[c.y][c.x] + 1;

                if (back < MAX_PATH) {
                    queue[back].x = nx;
                    queue[back].y = ny;
                    back++;
                }
            }
        }
    }
}

// Seeds all four centre cells at distance 0 and floods outward, so
// "distance to goal" is correct no matter which of the four centre
// cells the robot actually walks into.
void floodFillGoal(void)
{
    Cell queue[MAX_PATH];
    int back = 0;

    resetDistances();

    for (int gx = 3; gx <= 4; gx++) {
        for (int gy = 3; gy <= 4; gy++) {
            dist[gy][gx] = 0;
            queue[back].x = gx;
            queue[back].y = gy;
            back++;
        }
    }

    runFloodFromQueue(queue, back);
}

void floodFillTo(int targetX, int targetY)
{
    Cell queue[MAX_PATH];
    int back = 0;

    resetDistances();

    if (targetX < 0 || targetX >= MAZE_W || targetY < 0 || targetY >= MAZE_H) {
        return;
    }
}

```



```

    }

    dist[targetY][targetX] = 0;
    queue[back].x = targetX;
    queue[back].y = targetY;
    back++;

    runFloodFromQueue(queue, back);
}

// Walks downhill through the distance map from `start` to whichever
// zero-distance cell it leads to, recording each step into `path`.
// Returns how many cells are in the path (0 if there's no route).
int tracePath(Cell start, Cell path[])
{
    if (start.x < 0 || start.x >= MAZE_W || start.y < 0 || start.y >= MAZE_H) {
        return 0;
    }

    if (dist[start.y][start.x] == DIST_UNREACHABLE) {
        return 0;
    }

    Cell current = start;
    int pathLength = 0;
    path[pathLength++] = current;

    int safety = 0;
    while (dist[current.y][current.x] != 0 && safety < MAX_PATH) {
        int bestDist = dist[current.y][current.x];
        Cell next = current;
        int found = 0;

        for (int d = 0; d < 4; d++) {
            if (!canMove(current.x, current.y, (Direction)d)) {
                continue;
            }

            int nx = current.x + DX[d];
            int ny = current.y + DY[d];

            if (dist[ny][nx] < bestDist) {
                bestDist = dist[ny][nx];
                next.x = nx;
                next.y = ny;
                found = 1;
            }
        }

        if (!found) {
            break; // stuck - the map doesn't actually connect through here
        }

        current = next;

        if (pathLength < MAX_PATH) {
            path[pathLength++] = current;
        }

        safety++;
    }

    return pathLength;
}

```

```

// -----
// Control logic
// -----

// The centre "goal" is really a 2x2 block, so check all four cells
// rather than just the one GOAL_X/GOAL_Y anchor. (The original only
// checked (3,3), which meant the robot could walk into the centre
// via (3,4), (4,3) or (4,4) and never realise it had arrived.)
int atGoal(void)
{
    return (currentX == 3 || currentX == 4) && (currentY == 3 || currentY == 4);
}

int atStart(void)
{
    return currentX == START_X && currentY == START_Y;
}

// Reads the sensors, works out which walls that implies around the
// current cell, and records anything new in the map.
void updateWallMap(void)
{
    unsigned char previousWalls = walls[currentY][currentX];
    unsigned char pattern = readWallPattern();
    SensorAction action = classifySensorPattern(pattern);

    showSensorActionOnLEDs(action);

    if (frontGroupDetected(pattern)) {
        setWallBidirectional(currentX, currentY, heading);
    }

    if (leftGroupDetected(pattern)) {
        setWallBidirectional(currentX, currentY, turnLeft(heading));
    }

    if (rightGroupDetected(pattern)) {
        setWallBidirectional(currentX, currentY, turnRight(heading));
    }

    if (walls[currentY][currentX] != previousWalls) {
        ledPulse(LED_WALL, 50000); // new wall discovered
    }
}

// Plans one step from the current cell using the latest flood-fill
// map, turns to face it, double-checks for a wall, then drives.
// Returns 1 on success, 0 if the move couldn't be made.
int moveOneStep(void)
{
    Cell start = { currentX, currentY };
    Cell path[MAX_PATH];

    int pathLength = tracePath(start, path);

    if (pathLength < 2) {
        ledOn(LED_ERROR);
        motorStop();
        return 0;
    }

    applySensorCorrection(classifySensorPattern(readWallPattern()));

    Cell next = path[1];
    Direction targetDirection = cellToDirection(start, next);

```

```

    rotateToDirection(targetDirection);

    if (frontWallSensor()) {
        motorStop();
        ledPulse(LED_WALL, 200000);
        return 0;
    }

    driveForward();

    currentX = next.x;
    currentY = next.y;

    return 1;
}

// Drives a pre-computed path from start to finish without re-planning,
// used for the final speed run once the maze is fully known.
int executePath(Cell path[], int pathLength)
{
    if (pathLength < 2) {
        ledOn(LED_ERROR);
        return 0;
    }

    for (int i = 0; i < pathLength - 1; i++) {
        Direction targetDirection = cellToDirection(path[i], path[i + 1]);
        rotateToDirection(targetDirection);

        if (frontWallSensor()) {
            motorStop();
            ledOn(LED_ERROR);
            return 0;
        }

        driveForward();
    }

    return 1;
}

// -----
// Main
// -----

int main(void)
{
    sensors.mode(PullUp); // sensors are active-low, so pull the lines up
    ledsOff();
    motorStop();
    initialiseMazeArrays();
    floodFillGoal();

    while (phase != PHASE_COMPLETE) {
        switch (phase) {

            case PHASE_EXPLORE:
                updateWallMap();
                floodFillGoal();

                if (atGoal()) {
                    ledOn(LED_GOAL);
                    phase = PHASE_RETURN;
                    break;

```

```

    }

    moveOneStep();
    break;

case PHASE_RETURN:
    updateWallMap();
    floodFillTo(START_X, START_Y);

    if (atStart()) {
        ledOff(LED_GOAL);
        phase = PHASE_SPEEDRUN;
        break;
    }

    moveOneStep();
    break;

case PHASE_SPEEDRUN: {
    currentX = START_X;
    currentY = START_Y;
    heading = NORTH;

    floodFillGoal();

    Cell speedStart = { START_X, START_Y };
    Cell speedPath[MAX_PATH];
    int speedPathLength = tracePath(speedStart, speedPath);

    int ok = executePath(speedPath, speedPathLength);
    ledOn(ok ? LED_GOAL : LED_ERROR);

    phase = PHASE_COMPLETE;
    break;
}

case PHASE_COMPLETE:
    motorStop();
    break;
}

    wait_us(10000); // brief pause between control-loop iterations
}

motorStop();

while (1) {
    wait_us(1000000);
}
}

```